



Saad Mughal | F2023-009
Total Marks: 100
Due 3 June 2026

DevOps and Cloud Computing

Assignment 3

Kubernetes Orchestration & Full DevOps Pipeline



Course Lecturer: Muhammad Ali
Section: A

Table of Contents

Architecture Diagram.....	3
Part A - Git & Version Control.....	4
A1. Git Identity Configuration.....	4
A2. Repository Initialization & GitHub Push.....	4
A3. Branching Strategy.....	6
A4. Commit History.....	7
A5. .gitignore Configuration.....	8
Part B - Docker Containerization.....	9
B1. Dockerfiles.....	9
B2. Docker Compose - 3-Tier Stack.....	10
B3. Running Containers.....	10
B4. Custom Bridge Network.....	11
B5. Named Volume - MySQL Persistence.....	11
B6. Health Endpoint.....	12
B7. Volume Persistence After Restart.....	12
Part C - CI/CD with GitHub Actions.....	12
C1. Workflow File.....	12
C2. GitHub Secrets.....	13
C3. Pipeline Execution — Success.....	13
C4. DockerHub Images Published.....	13
Part D — Kubernetes Orchestration (55 Marks).....	14
D1. Minikube Setup & Namespace (10 Marks).....	14
D2. Service Types Explanation.....	14
D3. Persistent Storage - PV & PVC (10 Marks).....	15
D4. Secrets & ConfigMaps (10 Marks).....	15
D5. Scaling, Rolling Update & Rollback (10 Marks).....	16
Scaling to 3 Replicas.....	16
Rolling Update.....	16
Rollback.....	16
D6. End-to-End Testing & Verification (5 Marks).....	16
Curl Test via NodePort.....	16
Self-Healing.....	17
Database Persistence After MySQL Pod Deletion.....	17
YAML Manifest Explanations.....	17
1. namespace.yml.....	18
2. mysql-secret.yml.....	18
3. flask-configmap.yml.....	18

4. nginx-configmap.yml.....	18
5. mysql-pv.yml.....	18
6. mysql-pvc.yml.....	18
7. mysql-deployment.yml.....	19
8. mysql-service.yml.....	19
9. flask-deployment.yml.....	19
10. flask-service.yml.....	19
11. nginx-deployment.yml.....	19
12. nginx-service.yml.....	19
Conclusion.....	20

Assignment 3

Kubernetes Orchestration & Full DevOps Pipeline

Architecture Diagram

The diagram below illustrates the complete 3-tier DevOps pipeline. The system consists of an Nginx reverse proxy exposed via NodePort, a Flask REST API running with 3 replicas (ClusterIP), and a MySQL database with persistent storage (ClusterIP). GitHub Actions automates CI/CD and pushes Docker images to DockerHub, which Kubernetes pulls for deployments.

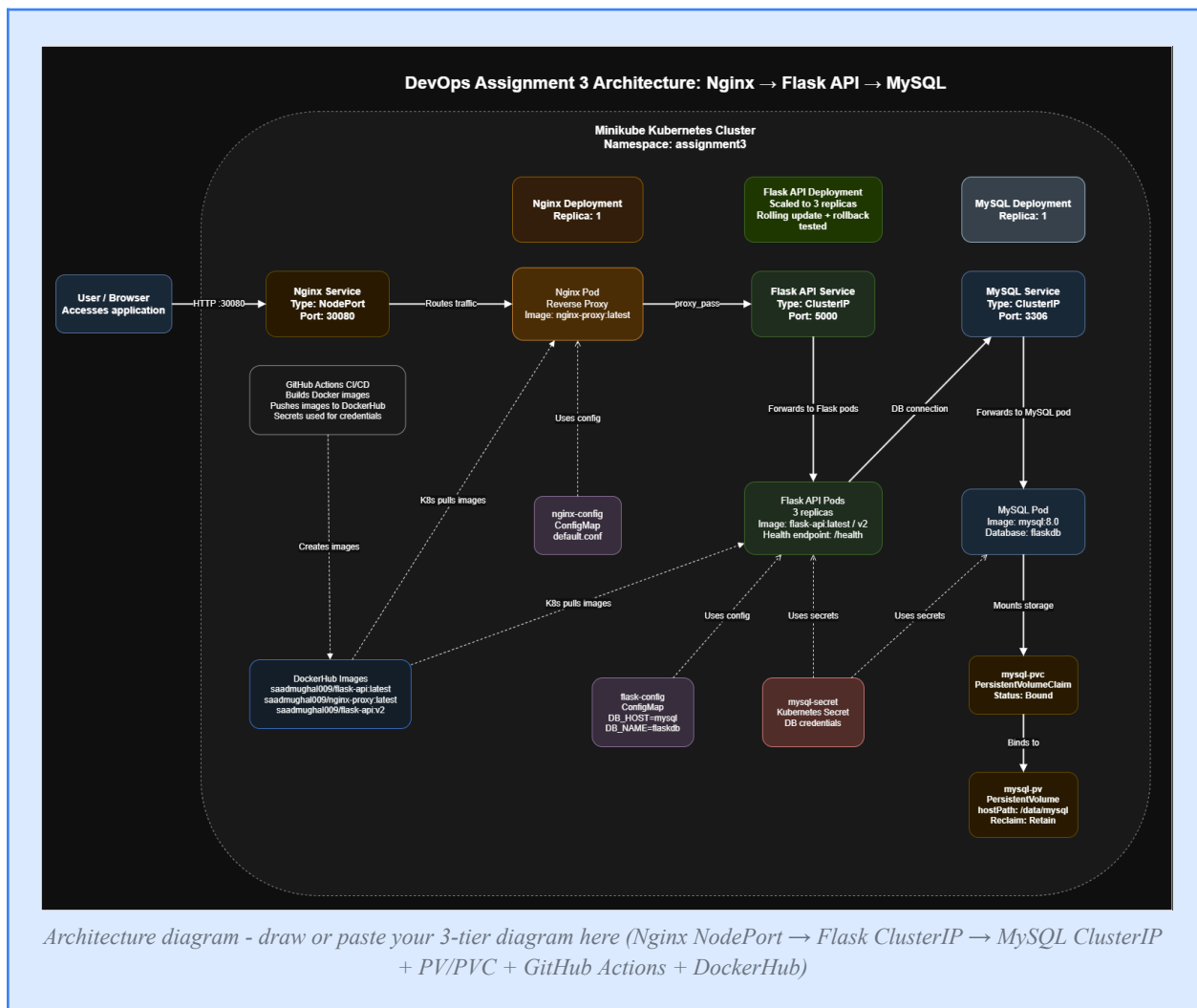


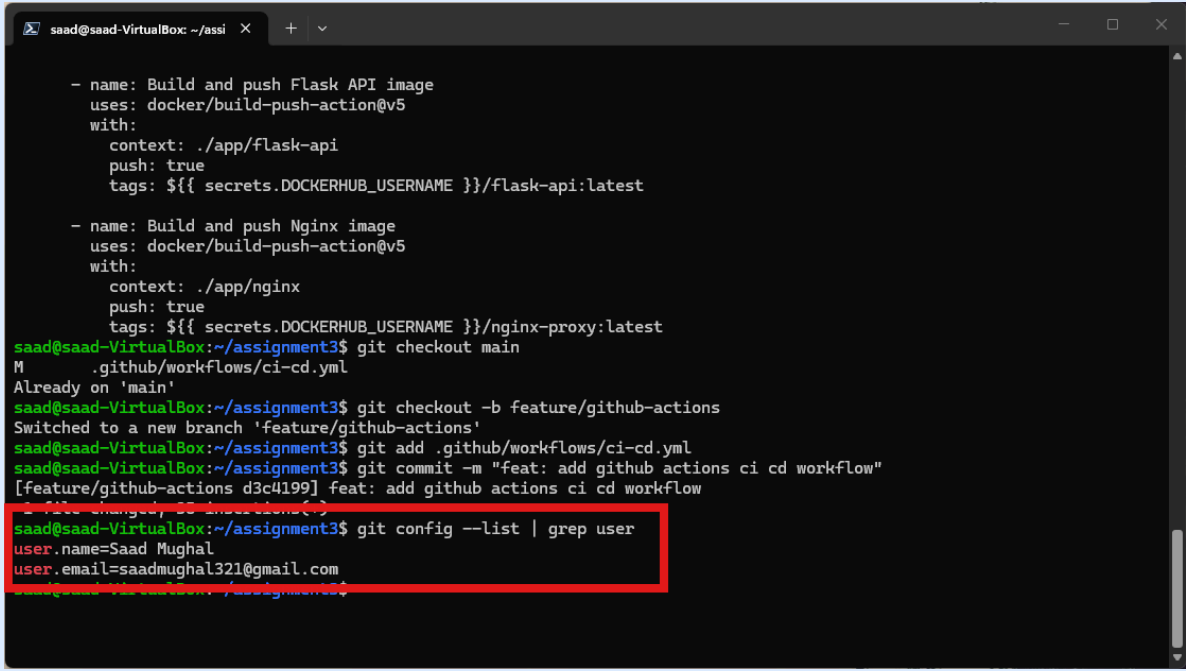
Figure 0: 3-Tier Architecture — Nginx → Flask API → MySQL on Minikube (Namespace: assignment3)

Part A - Git & Version Control

This section covers the Git repository setup, branching strategy, commit history, and GitHub integration for the assignment project.

A1. Git Identity Configuration

Git was configured with a user name and email before any commits were made using *git config --global user.name* and *git config --global user.email*. This ensures all commits are properly attributed.



```
saad@saad-VirtualBox: ~/assignment3
- name: Build and push Flask API image
  uses: docker/build-push-action@v5
  with:
    context: ./app/flask-api
    push: true
    tags: ${{ secrets.DOCKERHUB_USERNAME }}/flask-api:latest

- name: Build and push Nginx image
  uses: docker/build-push-action@v5
  with:
    context: ./app/nginx
    push: true
    tags: ${{ secrets.DOCKERHUB_USERNAME }}/nginx-proxy:latest

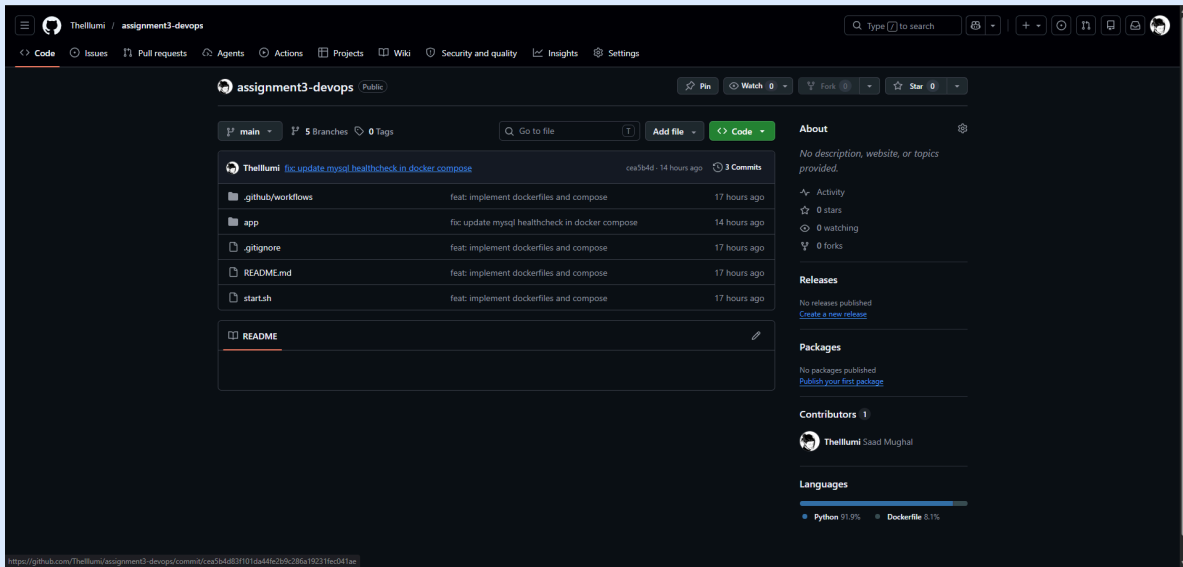
saad@saad-VirtualBox:~/assignment3$ git checkout main
M       .github/workflows/ci-cd.yml
Already on 'main'
saad@saad-VirtualBox:~/assignment3$ git checkout -b feature/github-actions
Switched to a new branch 'feature/github-actions'
saad@saad-VirtualBox:~/assignment3$ git add .github/workflows/ci-cd.yml
saad@saad-VirtualBox:~/assignment3$ git commit -m "feat: add github actions ci cd workflow"
[feature/github-actions d3c4199] feat: add github actions ci cd workflow
2 files changed, 55 insertions(+)
saad@saad-VirtualBox:~/assignment3$ git config --list | grep user
user.name=Saad Mughal
user.email=saadmughal321@gmail.com
```

part-a-git-config.png — git config --list | grep user

Figure 1: Git identity configuration showing user.name and user.email

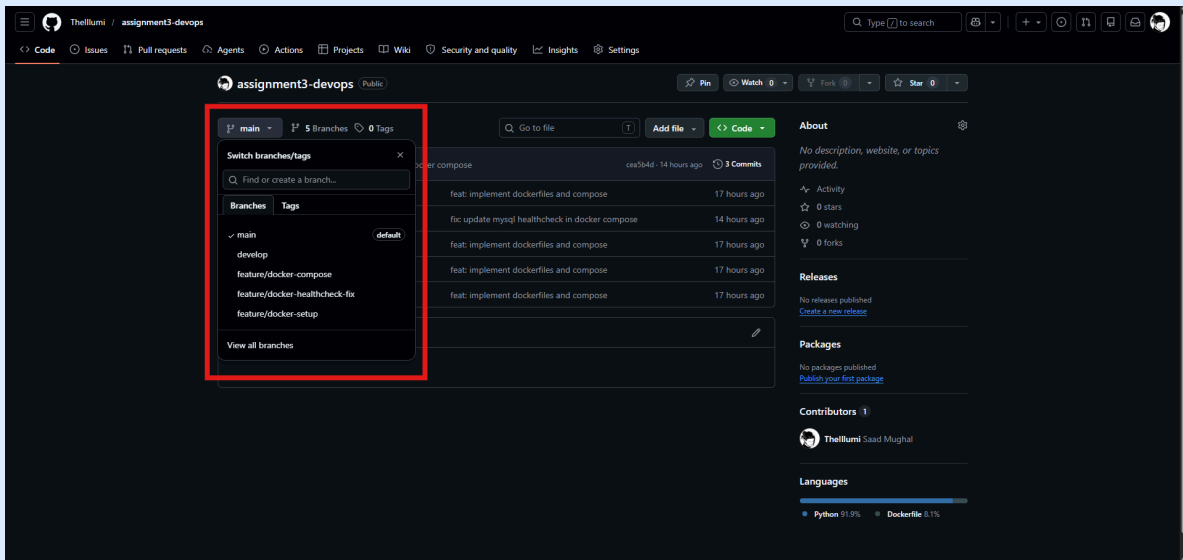
A2. Repository Initialization & GitHub Push

The project repository was initialized locally using *git init*, then linked to <https://github.com/TheIllumi/assignment3-devops>. All project files were committed and pushed to GitHub.



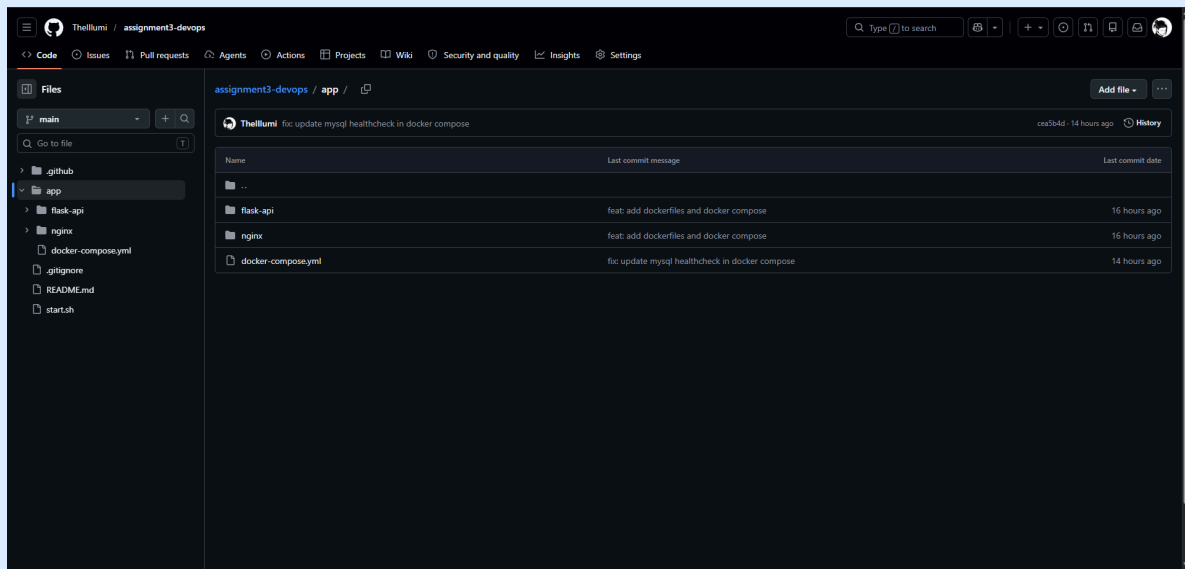
part-a-github-repo-1.png — GitHub repo main view

Figure 2: GitHub repository main page showing project files and structure



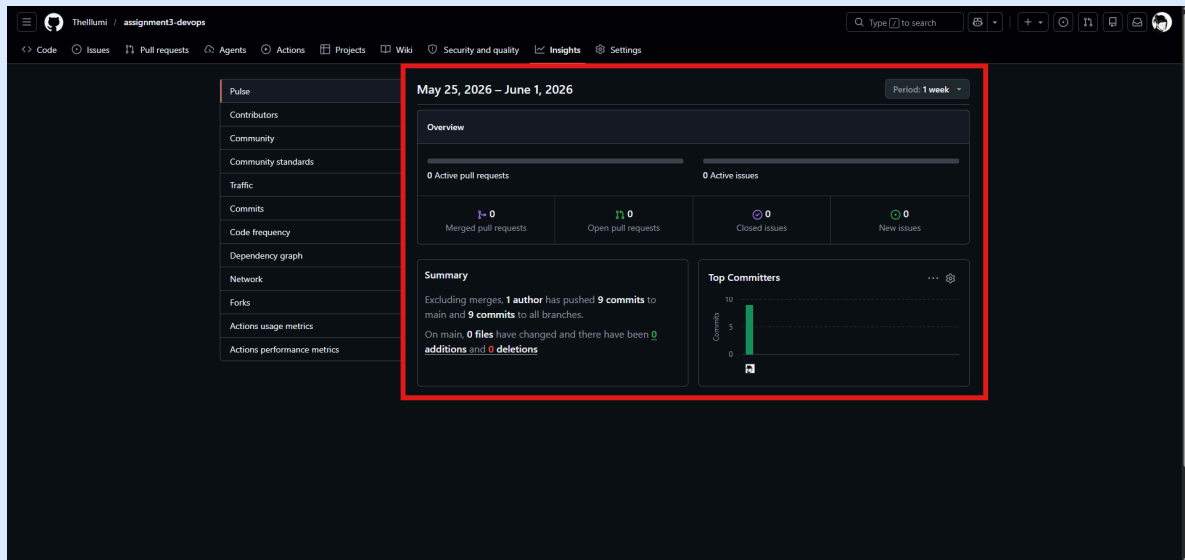
part-a-github-repo-2.png — GitHub repo commit count / branches

Figure 3: GitHub repository showing commit count and branch list



part-a-github-repo-3.png — GitHub repo additional view

Figure 4: GitHub repository — additional project view



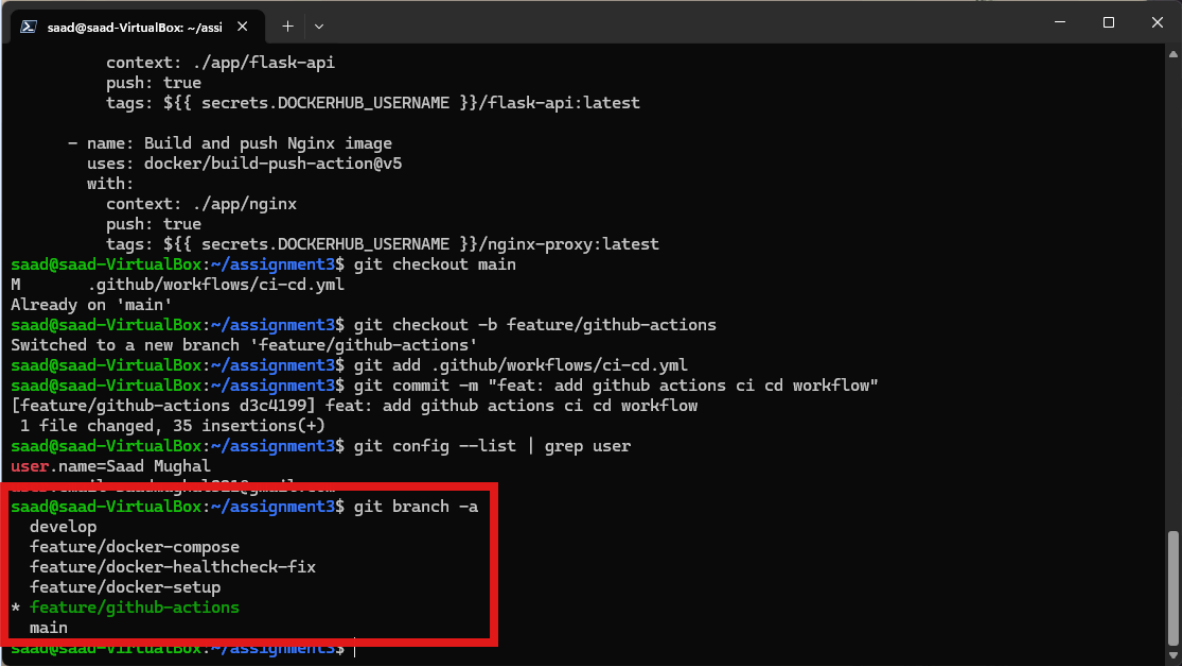
part-a-create-time.png — Repository creation timestamp

Figure 5: Repository creation time proof

A3. Branching Strategy

A structured Git branching strategy was followed throughout the project. Three branch types were used:

- **main** - production-ready, stable code only
- **develop** - integration branch for tested features
- **feature/*** - individual feature branches merged into develop



```

context: ./app/flask-api
push: true
tags: ${ secrets.DOCKERHUB_USERNAME }}/flask-api:latest

- name: Build and push Nginx image
  uses: docker/build-push-action@v5
  with:
    context: ./app/nginx
    push: true
    tags: ${ secrets.DOCKERHUB_USERNAME }}/nginx-proxy:latest
saad@saad-VirtualBox:~/assignment3$ git checkout main
M      .github/workflows/ci-cd.yml
Already on 'main'
saad@saad-VirtualBox:~/assignment3$ git checkout -b feature/github-actions
Switched to a new branch 'feature/github-actions'
saad@saad-VirtualBox:~/assignment3$ git add .github/workflows/ci-cd.yml
saad@saad-VirtualBox:~/assignment3$ git commit -m "feat: add github actions ci cd workflow"
[feature/github-actions d3c4199] feat: add github actions ci cd workflow
1 file changed, 35 insertions(+)
saad@saad-VirtualBox:~/assignment3$ git config --list | grep user
user.name=Saad Mughal
saad@saad-VirtualBox:~/assignment3$ git branch -a
  develop
  feature/docker-compose
  feature/docker-healthcheck-fix
  feature/docker-setup
* feature/github-actions
  main
saad@saad-VirtualBox:~/assignment3$

```

part-a-git-branches.png — git branch -a output

Figure 6: All Git branches — main, develop, and feature branches visible

A4. Commit History

A minimum of 10 meaningful commits were made throughout the development process, each with a descriptive commit message. The commit log demonstrates a clean, professional Git workflow.


```
saad@saad-VirtualBox: ~/assignment3$ git rev-list --count --all
10
saad@saad-VirtualBox:~/assignment3$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
saad@saad-VirtualBox:~/assignment3$ cd ~/assignment3
saad@saad-VirtualBox:~/assignment3$ git log --oneline --graph --all --decorate -15
* 6a54ce4 (HEAD -> main, origin/main) docs: document kubernetes deployment
* ac103d1 docs: describe project components
* d834826 docs: add project overview to readme
* f45b5a5 chore: add startup script for kubernetes deployment
* a9d0f01 feat: add kubernetes manifests
* 1caaf91 feat: add GitHub Actions CI/CD pipeline
| * d3c4199 (feature/github-actions) feat: add github actions ci cd workflow
|/
* cea5b4d (origin/feature/docker-healthcheck-fix, origin/develop, feature/docker-healthcheck-fix, develop) fix: update m
ysql healthcheck in docker compose
* 0ec0c7f (origin/feature/docker-compose, feature/docker-compose) feat: add dockerfiles and docker compose
* e75eaba (origin/feature/docker-setup, feature/docker-setup) feat: implement dockerfiles and compose
```

part-a-git-log.png — git log --oneline -15 output

Figure 7: Git commit log showing 10+ descriptive commits

A5. .gitignore Configuration

A `.gitignore` file was added to prevent sensitive and unnecessary files from being tracked. The file excludes: `__pycache__`, `.env`, `.vscode`, `.DS_Store`.

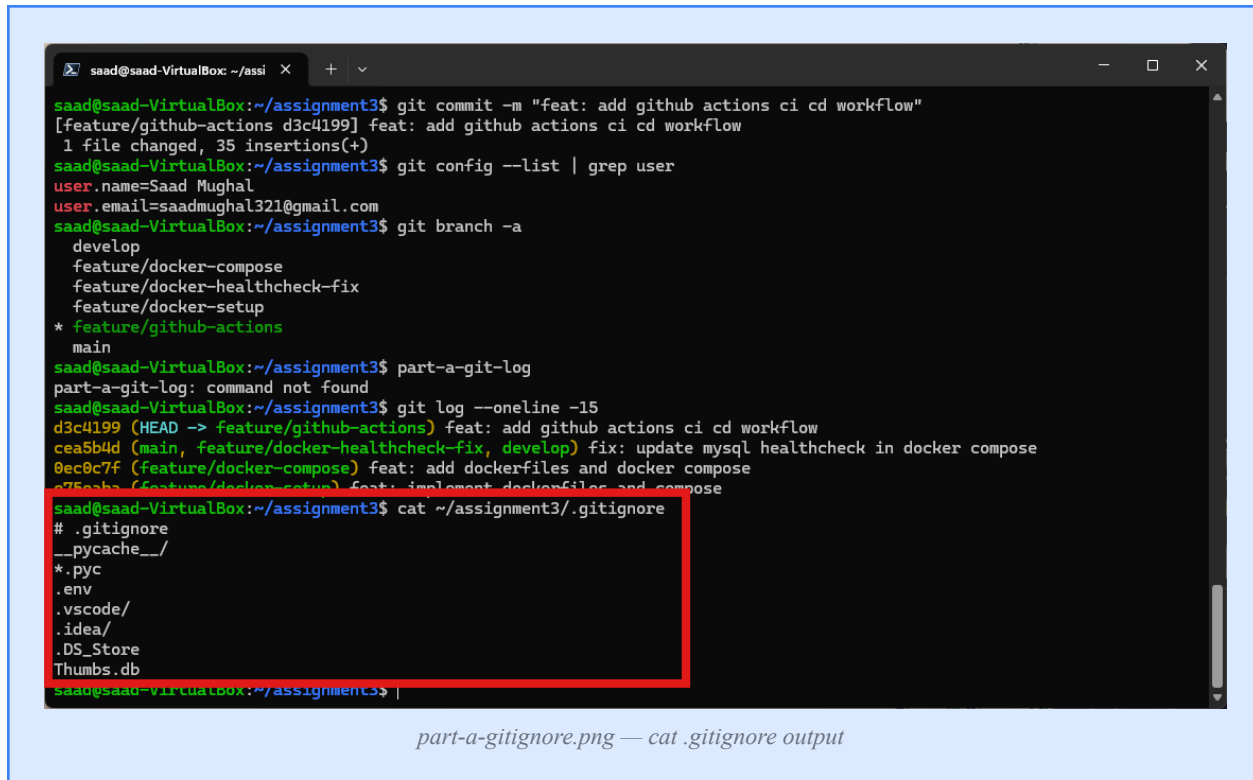


Figure 8: .gitignore contents — excludes __pycache__/, .env, .vscode/, .DS_Store

Part B - Docker Containerization

This section covers the Dockerfiles, Docker Compose configuration, custom bridge network, named volumes, health endpoint verification, and data persistence testing.

B1. Dockerfiles

Two Dockerfiles were created: one for the Flask REST API and one for the Nginx reverse proxy. The Flask Dockerfile uses a Python base image, installs dependencies from requirements.txt, and runs the API with Gunicorn. The Nginx Dockerfile uses the official Nginx base image and copies a custom nginx.conf for reverse proxy routing.

```
saad@saad-VirtualBox: ~/assignment3/app$ docker compose up -d
WARN[0000] /home/saad/assignment3/app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 4/4
✓Network app_app-network Created 0.1s
✓Container app-mysql-1 Healthy 16.7s
✓Container app-flask-api-1 Started 16.7s
✓Container app-nginx-1 Started 17.1s
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/api/items
[{"created_at":"2026-06-01 05:52:14","description":"Docker test","id":2,"name":"Test Item"}, {"created_at":"2026-05-31 17
saad@saad-VirtualBox:~/assignment3/app$ ls ~/assignment3/app/nginx/Dockerfile
/home/saad/assignment3/app/nginx/Dockerfile
saad@saad-VirtualBox:~/assignment3/app$ ls ~/assignment3/app/flask-api/Dockerfile
/home/saad/assignment3/app/flask-api/Dockerfile
saad@saad-VirtualBox:~/assignment3/app$ ls ~/assignment3/app/docker-compose.yml
/home/saad/assignment3/app/docker-compose.yml
saad@saad-VirtualBox:~/assignment3/app$ cat ~/assignment3/app/nginx/Dockerfile
FROM nginx:alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/conf.d/default.conf
saad@saad-VirtualBox:~/assignment3/app$ cat ~/assignment3/app/flask-api/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

part-b-dockerfiles.png — Flask and Nginx Dockerfile contents

Figure 9: Dockerfiles for Flask API and Nginx reverse proxy

B2. Docker Compose - 3-Tier Stack

A *docker-compose.yml* file orchestrates all three services — Nginx, Flask API, and MySQL — on a single custom bridge network. The compose file defines service dependencies, environment variables for MySQL credentials, health check intervals, and volume mounts.

B3. Running Containers

All three containers were started using *docker compose up -d*. The screenshot below confirms all three services are running with STATUS: Up.

```
saad@saad-VirtualBox: ~/assi x + v
.env
.vscode/
.idea/
.DS_Store
Thumbs.db
saad@saad-VirtualBox:~/assignment3$ cd ~/assignment3/app
saad@saad-VirtualBox:~/assignment3/app$ docker compose up -d
WARN[0000] /home/saad/assignment3/app/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 3/3
✓Container app-mysql-1 Healthy 0.6s
✓Container app-flask-api-1 Running 0.0s
✓Container app-nginx-1 Running 0.0s
saad@saad-VirtualBox:~/assignment3/app$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
37527dcaaf90	app-nginx	"/docker-entrypoint.s..."	13 hours ago	Up 13 hours	0.0
cb1f9a57bc6f	app-flask-api	"python app.py"	13 hours ago	Up 13 hours	500
cade1062ff33	mysql:8.0	"docker-entrypoint.s..."	13 hours ago	Up 13 hours (healthy)	330
dcce7e0c10b6	gcr.io/k8s-minikube/kicbase:v0.0.50	"/usr/local/bin/entr..."	18 hours ago	Up 18 hours	127

part-b-docker-ps.png — docker ps output

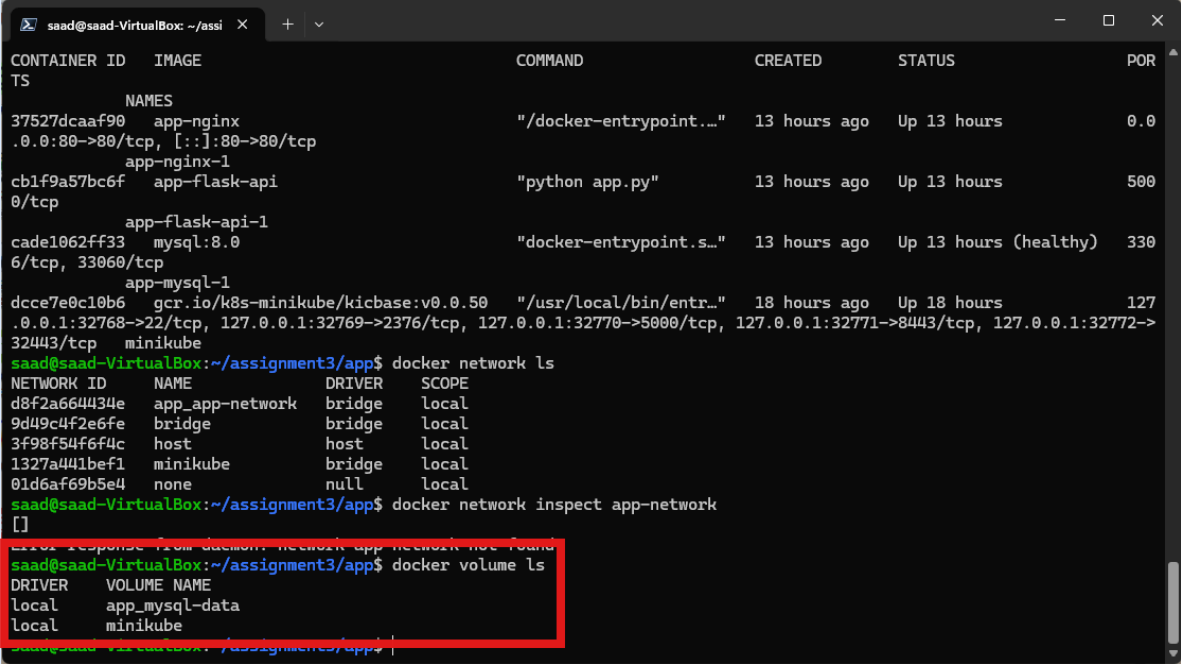
Figure 10: All 3 containers running — nginx, flask-api, mysql (STATUS: Up)

B4. Custom Bridge Network

A custom Docker bridge network named *app-network* was created to allow the three containers to communicate by service name (DNS resolution). This isolates the application network from the default Docker bridge.

B5. Named Volume - MySQL Persistence

A named Docker volume *mysql-data* was defined in *docker-compose.yml* to persist MySQL database files across container restarts. Data is not lost when containers are stopped and restarted.



The screenshot shows a terminal window with the following content:

```
saad@saad-VirtualBox: ~/assi
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
37527dcaaf90	app-nginx	"/docker-entrypoint..."	13 hours ago	Up 13 hours	0.0
cb1f9a57bc6f	app-flask-api	"python app.py"	13 hours ago	Up 13 hours	500
cade1062ff33	mysql:8.0	"docker-entrypoint.s..."	13 hours ago	Up 13 hours (healthy)	330
dcce7e0c10b6	gcr.io/k8s-minikube/kicbase:v0.0.50	"/usr/local/bin/entr..."	18 hours ago	Up 18 hours	127

```
saad@saad-VirtualBox:~/assignment3/app$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
d8f2a664434e	app_app-network	bridge	local
9d49c4f2e6fe	bridge	bridge	local
3f98f54f6f4c	host	host	local
1327a441bef1	minikube	bridge	local
01d6af69b5e4	none	null	local

```
saad@saad-VirtualBox:~/assignment3/app$ docker network inspect app-network
```

```
[ ]
```

```
Error response from daemon: network app_network not found
```

```
saad@saad-VirtualBox:~/assignment3/app$ docker volume ls
```

DRIVER	VOLUME NAME
local	app_mysql-data
local	minikube

part-b-docker-volumes.png — docker volume ls output

Figure 13: Named volume 'mysql-data' (or assignment3_mysql-data) listed

B6. Health Endpoint

The Flask API exposes a */health* endpoint accessible through the Nginx reverse proxy, verifying the full Nginx → Flask communication path.

```
saad@saad-VirtualBox: ~/assi x + v
NAMES
37527dcaaf90 app-nginx "/docker-entrypoint..." 13 hours ago Up 13 hours 0.0
.0.0:80->80/tcp, [::]:80->80/tcp
app-nginx-1
cb1f9a57bc6f app-flask-api "python app.py" 13 hours ago Up 13 hours 500
0/tcp
app-flask-api-1
cade1062ff33 mysql:8.0 "docker-entrypoint.s..." 13 hours ago Up 13 hours (healthy) 330
6/tcp, 33060/tcp
app-mysql-1
dcce7e0c10b6 gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 18 hours ago Up 18 hours 127
.0.0.1:32768->22/tcp, 127.0.0.1:32769->2376/tcp, 127.0.0.1:32770->5000/tcp, 127.0.0.1:32771->8443/tcp, 127.0.0.1:32772->
32443/tcp minikube
saad@saad-VirtualBox:~/assignment3/app$ docker network ls
NETWORK ID NAME DRIVER SCOPE
d8f2a664434e app_app-network bridge local
9d49c4f2e6fe bridge bridge local
3f98f54f6f4c host host local
1327a441bef1 minikube bridge local
01d6af69b5e4 none null local
saad@saad-VirtualBox:~/assignment3/app$ docker network inspect app-network
[]
Error response from daemon: network app-network not found
saad@saad-VirtualBox:~/assignment3/app$ docker volume ls
DRIVER VOLUME NAME
local app_mysql-data
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/health
{"service":"flask-api","status":"healthy"}
saad@saad-VirtualBox:~/assignment3/app$
```

part-b-health-check.png — curl http://localhost/health response

Figure 14: Health check response — {"service":"flask-api","status":"healthy"}

B7. Volume Persistence After Restart

An item was created via POST API, then Docker Compose was brought down with *docker compose down* and back up. The subsequent GET request confirmed the data was still present, proving the named volume correctly persists MySQL data.

```

saad@saad-VirtualBox: ~/assi
cb1f9a57bc6f  app-flask-api      "python app.py"      13 hours ago  Up 13 hours      500
0/tcp
    app-flask-api-1
cade1062ff33  mysql:8.0          "docker-entrypoint.s..." 13 hours ago  Up 13 hours (healthy) 330
6/tcp, 33060/tcp
    app-mysql-1
dcce7e0c10b6  gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 18 hours ago  Up 18 hours      127
.0.0.1:32768->22/tcp, 127.0.0.1:32769->2376/tcp, 127.0.0.1:32770->5000/tcp, 127.0.0.1:32771->8443/tcp, 127.0.0.1:32772->
32443/tcp  minikube
saad@saad-VirtualBox:~/assignment3/app$ docker network ls
NETWORK ID      NAME             DRIVER          SCOPE
d8f2a664434e   app_app-network  bridge          local
9d49c4f2e6fe   bridge          bridge          local
3f98f54f6f4c   host            host            local
1327a441bef1   minikube        bridge          local
01d6af69b5e4   none            null            local
saad@saad-VirtualBox:~/assignment3/app$ docker network inspect app-network
[]
Error response from daemon: network app-network not found
saad@saad-VirtualBox:~/assignment3/app$ docker volume ls
DRIVER          VOLUME NAME
local           app_mysql-data
local           minikube
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/health
{"service": "flask-api", "status": "healthy"}
saad@saad-VirtualBox:~/assignment3/app$ curl -X POST http://localhost/api/items \
-H "Content-Type: application/json" \
-d '{"name": "Test Item", "description": "Docker test"}'
{"description": "Docker test", "id": 2, "name": "Test Item"}
saad@saad-VirtualBox:~/assignment3/app$

```

part-b-create-item.png — POST request creating a new item in the Flask API

Figure 15: Item successfully created through the API, confirming Flask and MySQL connection

```

saad@saad-VirtualBox: ~/assi
local minikube
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/health
{"service": "flask-api", "status": "healthy"}
saad@saad-VirtualBox:~/assignment3/app$ curl -X POST http://localhost/api/items \
-H "Content-Type: application/json" \
-d '{"name": "Test Item", "description": "Docker test"}'
{"description": "Docker test", "id": 2, "name": "Test Item"}
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/api/items
[{"created_at": "2026-06-01 05:52:14", "description": "Docker test", "id": 2, "name": "Test Item"}, {"created_at": "2026-05-31 17
:01:53", "description": "This is a test", "id": 1, "name": "Test Item"}]
saad@saad-VirtualBox:~/assignment3/app$ docker compose down
WARN[0000] /home/saad/assignment3/app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pleas
e remove it to avoid potential confusion
[+] down 4/4
✓Container app-nginx-1      Removed      1.1s
✓Container app-flask-api-1  Removed      10.6s
✓Container app-mysql-1      Removed      4.1s
✓Network app_app-network    Removed      0.8s
saad@saad-VirtualBox:~/assignment3/app$ docker compose up -d
WARN[0000] /home/saad/assignment3/app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pleas
e remove it to avoid potential confusion
[+] up 4/4
✓Network app_app-network    Created      0.1s
✓Container app-mysql-1      Healthy      16.7s
✓Container app-flask-api-1  Started      16.7s
✓Container app-nginx-1      Started      17.1s
saad@saad-VirtualBox:~/assignment3/app$ curl http://localhost/api/items
[{"created_at": "2026-06-01 05:52:14", "description": "Docker test", "id": 2, "name": "Test Item"}, {"created_at": "2026-05-31 17
:01:53", "description": "This is a test", "id": 1, "name": "Test Item"}]

```

part-b-volume-persistence.png — curl after docker compose restart

Figure 16: Items still present after docker compose down + up — volume persistence confirmed

Part C - CI/CD with GitHub Actions

This section documents the automated CI/CD pipeline built with GitHub Actions, which builds and pushes Docker images to DockerHub on every push to the main branch.

C1. Workflow File

The CI/CD workflow is defined in `.github/workflows/ci-cd.yml`. It triggers on every push to main and performs these steps:

- Checks out the repository code
- Logs into DockerHub using repository secrets (no hardcoded credentials)
- Builds and tags Flask API image as `saadmughal009/flask-api:latest`
- Builds and tags Nginx image as `saadmughal009/nginx-proxy:latest`
- Pushes both images to DockerHub

C2. GitHub Secrets

DockerHub credentials are stored as encrypted GitHub Secrets (`DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`) to prevent exposure of sensitive data in the workflow file.

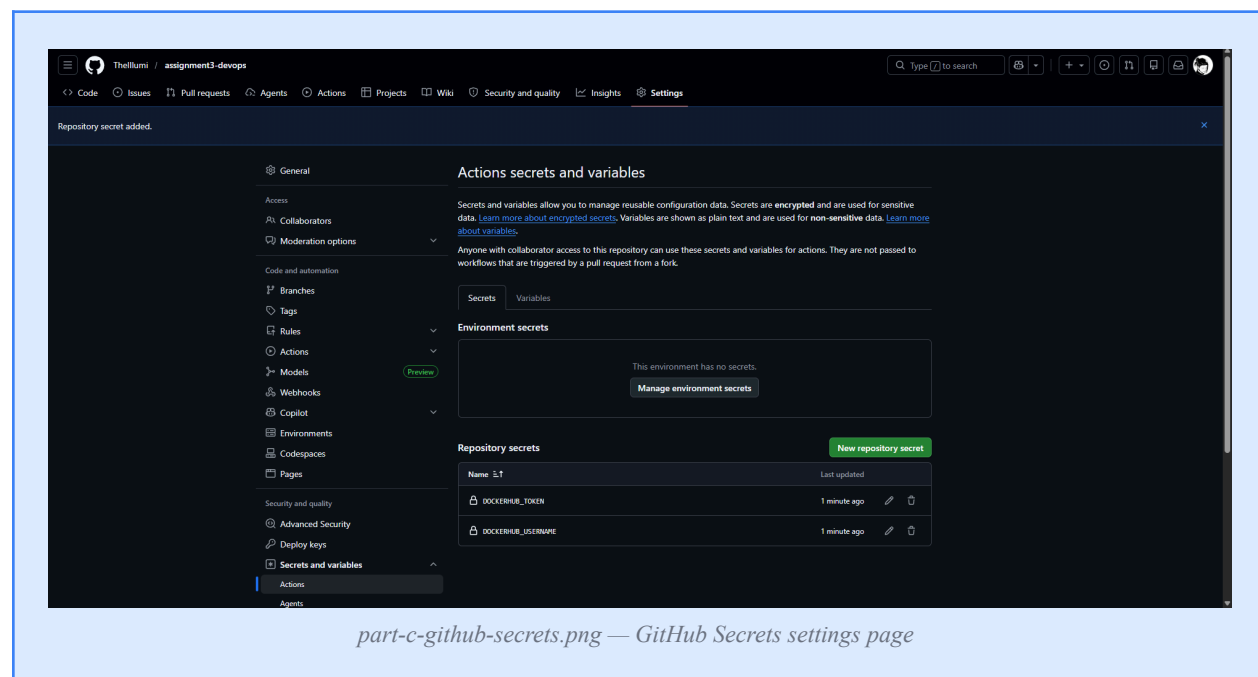


Figure 16: GitHub repository secrets — `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` configured

C3. Pipeline Execution - Success

The GitHub Actions pipeline ran successfully on push to main. The green checkmark in the Actions tab confirms all workflow steps completed without errors.

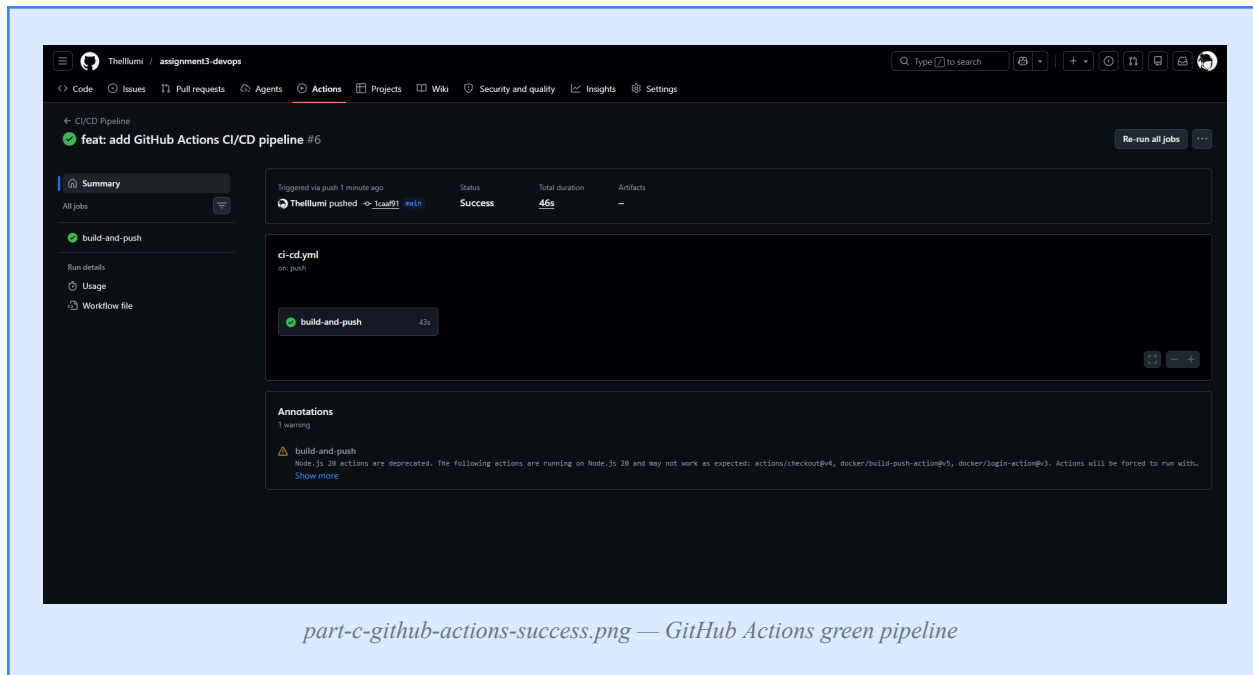


Figure 17: GitHub Actions workflow — all steps passed (green checkmarks)

C4. DockerHub Images Published

Following the successful pipeline run, both Docker images are visible on DockerHub under account *saadmughal009*:

- *saadmughal009/flask-api:latest* — Flask REST API
- *saadmughal009/nginx-proxy:latest* — Nginx reverse proxy
- *saadmughal009/flask-api:v2* — Built for rolling update (Part D5)

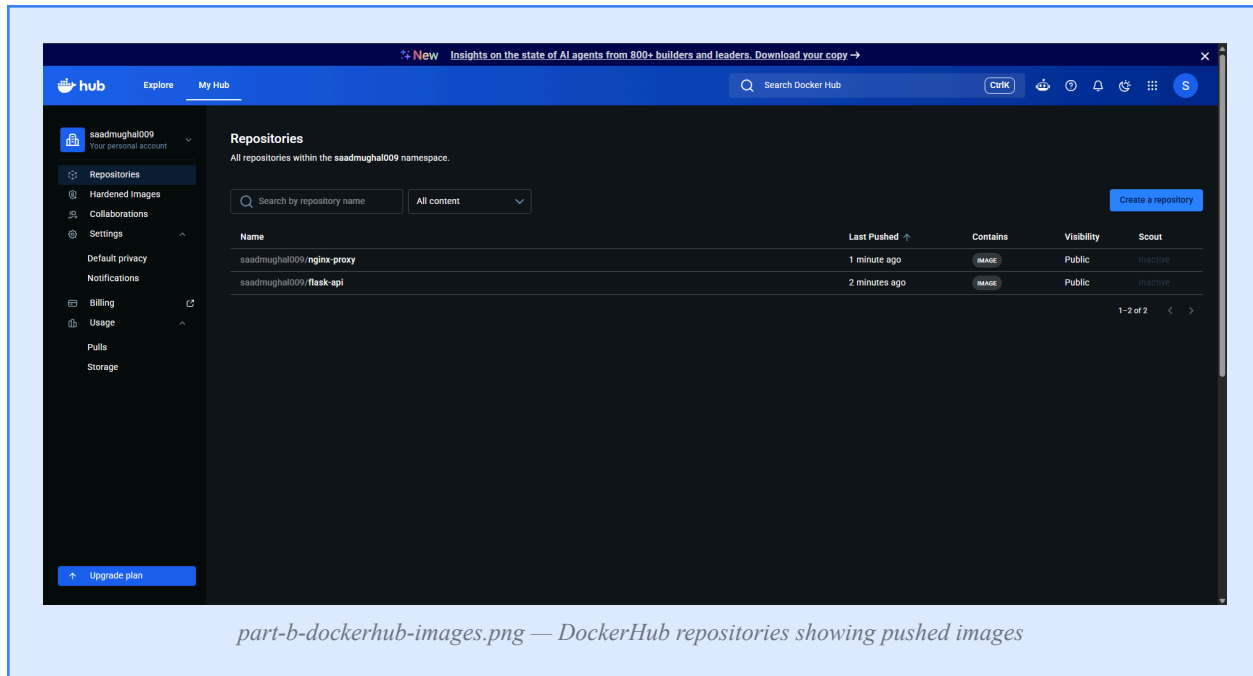


Figure 18: DockerHub — flask-api and nginx-proxy images pushed by CI/CD pipeline

Part D - Kubernetes Orchestration

All Kubernetes resources were deployed to a Minikube cluster using the Docker driver on Ubuntu 22.04. Namespace *assignment3* was used to isolate all resources. All manifests are located in the *k8s/* directory.

D1. Minikube Setup & Namespace

Minikube was started with *minikube start --driver=docker*. Namespace *assignment3* was created via *kubectl apply -f k8s/namespace.yml*. All three deployments — MySQL, Flask API, and Nginx — were applied and verified running.

```
nothing to commit, working tree clean
saad@saad-VirtualBox: ~/assignment3$ git log --oneline -5
1caaf91 (HEAD -> main, origin/main) feat: add GitHub Actions CI/CD pipeline
cea5b4d (origin/feature/docker-healthcheck-fix, origin/develop, feature/docker-healthcheck-fix, develop) fix: update mysql healthcheck in docker compose
0ec0c7f (origin/feature/docker-compose, feature/docker-compose) feat: add dockerfiles and docker compose
0f5ebc6 (origin/feature/docker-setup, feature/docker-setup) feat: implement dockerfiles and compose

saad@saad-VirtualBox:~/assignment3$ minikube start --driver=docker --memory=2048 --cpus=2
🐳 minikube v1.38.1 on Ubuntu 22.04 (vbox/amd64)
🔧 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📶 Pulling base image v0.0.50 ...
🔄 Updating the running docker "minikube" container ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
👉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
saad@saad-VirtualBox:~/assignment3$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

saad@saad-VirtualBox:~/assignment3$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready     control-plane 19h   v1.35.1
```

part-d1-minikube-status.png — minikube status output

Figure 19: Minikube status — host, kubelet, apiserver all Running

```
service/flask-api created
saad@saad-VirtualBox:~/assignment3$ kubectl apply -f k8s/nginx-deployment.yml
deployment.apps/nginx created
saad@saad-VirtualBox:~/assignment3$ kubectl apply -f k8s/nginx-service.yml
service/nginx created
saad@saad-VirtualBox:~/assignment3$ kubectl wait --for=condition=Ready pod --all -n assignment3 --timeout=180s
pod/flask-api-7f56c46ddd-njl9c condition met
pod/mysql-64d85fb859-442l9 condition met
pod/nginx-6bc4679456-sc4bb condition met

saad@saad-VirtualBox:~/assignment3$ kubectl get all -n assignment3
NAME                                READY   STATUS    RESTARTS   AGE
pod/flask-api-7f56c46ddd-njl9c      1/1     Running   0           72s
pod/mysql-64d85fb859-442l9          1/1     Running   0           3m5s
pod/nginx-6bc4679456-sc4bb          1/1     Running   0           59s

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP    PORT(S)          AGE
service/flask-api                    ClusterIP      10.99.56.128    <none>          5000/TCP          66s
service/mysql                        ClusterIP      10.102.71.94    <none>          3306/TCP          3m
service/nginx                        NodePort       10.102.198.169  <none>          80:30080/TCP      54s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api            1/1     1             1           72s
deployment.apps/mysql                1/1     1             1           3m5s
deployment.apps/nginx                 1/1     1             1           59s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/flask-api-7f56c46ddd 1          1          1       72s
replicaset.apps/mysql-64d85fb859      1          1          1       3m5s
replicaset.apps/nginx-6bc4679456      1          1          1       59s
```

part-d1-kubectl-get-all.png — kubectl get all -n assignment3

Figure 20: kubectl get all -n assignment3 — pods, services, deployments all Running/Active

D2. Service Types Explanation

Three Kubernetes service types were used, each selected for a specific architectural reason:

Service	Type	Port	Reason
MySQL	ClusterIP	3306	Internal-only. The database must not be reachable from outside the cluster. Only Flask API communicates with MySQL over the cluster network.
Flask API	ClusterIP	5000	Internal-only. Flask is accessed only by Nginx within the cluster. External exposure would bypass the reverse proxy layer.
Nginx	NodePort	30080	External access required. NodePort maps port 80 to node port 30080, allowing curl/browser access via <code>http://\$(minikube ip):30080</code> .

Table 1: Kubernetes service types and justification

D3. Persistent Storage - PV & PVC

A PersistentVolume (PV) named *mysql-pv* was created with 1Gi capacity using a *hostPath* mount on the Minikube node. A PersistentVolumeClaim (PVC) named *mysql-pvc* claims that storage. The MySQL deployment mounts this PVC at */var/lib/mysql*, ensuring data survives pod deletions.

```

saad@saad-VirtualBox: ~/assignment3$ kubectl get all -n assignment3
NAME                                READY   STATUS    RESTARTS   AGE
pod/flask-api-7f56c46ddd-njl9c     1/1     Running   0           72s
pod/mysql-64d85fb859-442l9         1/1     Running   0           3m5s
pod/nginx-6bc4679456-sc4bb         1/1     Running   0           59s

NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/flask-api                   ClusterIP           10.99.56.128    <none>            5000/TCP          66s
service/mysql                       ClusterIP           10.102.71.94    <none>            3306/TCP          3m
service/nginx                       NodePort            10.102.198.169  <none>            80:30080/TCP      54s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api           1/1     1             1           72s
deployment.apps/mysql               1/1     1             1           3m5s
deployment.apps/nginx               1/1     1             1           59s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/flask-api-7f56c46ddd 1          1         1       72s
replicaset.apps/mysql-64d85fb859      1          1         1       3m5s
replicaset.apps/nginx-6bc4679456     1          1         1       59s

saad@saad-VirtualBox:~/assignment3$ kubectl get pv
NAME                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                STORAGECLASS   VOLUMEATTRIBUTESCL
mysql-pv            1Gi        RWO            Retain           Bound    assignment3/mysql-pvc manual           <unset>

saad@saad-VirtualBox:~/assignment3$ kubectl get pvc -n assignment3
NAME                STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
mysql-pvc           Bound    mysql-pv  1Gi        RWO            manual         <unset>                4m23s

```

part-d3-pvc-bound.png — kubectl get pv and pvc showing Bound status

Figure 21: PersistentVolume and PersistentVolumeClaim — STATUS: Bound

D4. Secrets & ConfigMaps

MySQL credentials are stored in a Kubernetes Secret named *mysql-secret* — values are Base64-encoded and never hardcoded. Non-sensitive configuration is stored in two ConfigMaps:

- *flask-config* — Flask database connection settings (host, DB name, port)
- *nginx-config* — Nginx reverse proxy configuration for routing to Flask

```

saad@saad-VirtualBox: ~/assi
service/flask-api ClusterIP 10.99.56.128 <none> 5000/TCP 66s
service/mysql ClusterIP 10.102.71.94 <none> 3306/TCP 3m
service/nginx NodePort 10.102.198.169 <none> 80:30080/TCP 54s

NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/flask-api 1/1 1 1 72s
deployment.apps/mysql 1/1 1 1 3m5s
deployment.apps/nginx 1/1 1 1 59s

NAME DESIRED CURRENT READY AGE
replicaset.apps/flask-api-7f56c46ddd 1 1 1 72s
replicaset.apps/mysql-64d85fb859 1 1 1 3m5s
replicaset.apps/nginx-6bc4679456 1 1 1 59s
saad@saad-VirtualBox:~/assignment3$ kubectl get pv
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM STORAGECLASS VOLUMEATTRIBUTESCL
ASS REASON AGE
mysql-pv 1Gi RWO Retain Bound assignment3/mysql-pvc manual <unset>
4m17s
saad@saad-VirtualBox:~/assignment3$ kubectl get pvc -n assignment3
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE
mysql-pvc Bound mysql-pv 1Gi RWO manual <unset> 4m23s
saad@saad-VirtualBox:~/assignment3$ kubectl get secrets -n assignment3
NAME TYPE DATA AGE
mysql-secret Opaque 4 5m49s
saad@saad-VirtualBox:~/assignment3$ kubectl get configmaps -n assignment3
NAME DATA AGE
flask-config 3 5m49s
kube-root-ca.crt 1 14m
nginx-config 1 5m43s

```

part-d4-secrets-configmaps.png — kubectl get secrets and configmaps

Figure 22: Kubernetes Secret and ConfigMaps successfully applied in namespace assignment3

D5. Scaling, Rolling Update & Rollback

Scaling to 3 Replicas

The Flask API deployment was scaled to 3 replicas using *kubectl scale deployment flask-api --replicas=3 -n assignment3*. Kubernetes distributed the 3 pods across the node, ensuring high availability.

```
mysql-secret Opaque 4 5m49s
saad@saad-VirtualBox: ~/assignment3$ kubectl get configmaps -n assignment3
NAME DATA AGE
flask-config 3 5m49s
kube-root-ca.crt 1 14m
nginx-config 1 5m43s
saad@saad-VirtualBox:~/assignment3$ MINK_IP=$(minikube ip)
echo "App URL: http://${MINK_IP}:30080"
curl http://${MINK_IP}:30080/health
App URL: http://192.168.49.2:30080
{"service": "flask-api", "status": "healthy"}
saad@saad-VirtualBox:~/assignment3$ curl -X POST http://${MINK_IP}:30080/api/items \
-H "Content-Type: application/json" \
-d '{"name": "Kubernetes Item", "description": "Created on K8s"}'
{"description": "Created on K8s", "id": 1, "name": "Kubernetes Item"}
saad@saad-VirtualBox:~/assignment3$ curl http://${MINK_IP}:30080/api/items
{"created_at": "2026-06-01 08:16:22", "description": "Created on K8s", "id": 1, "name": "Kubernetes Item"}
saad@saad-VirtualBox:~/assignment3$ kubectl scale deployment flask-api --replicas=3 -n assignment3
deployment.apps/flask-api scaled
saad@saad-VirtualBox:~/assignment3$ kubectl get pods -n assignment3 -l app=flask-api
NAME READY STATUS RESTARTS AGE
flask-api-7f56c46ddd-ldctk 0/1 ContainerCreating 0 6s
flask-api-7f56c46ddd-njl9c 1/1 Running 0 10m
flask-api-7f56c46ddd-x8l4s 0/1 ContainerCreating 0 6s
saad@saad-VirtualBox:~/assignment3$ kubectl get pods -n assignment3 -l app=flask-api
NAME READY STATUS RESTARTS AGE
flask-api-7f56c46ddd-ldctk 1/1 Running 0 81s
flask-api-7f56c46ddd-njl9c 1/1 Running 0 11m
flask-api-7f56c46ddd-x8l4s 1/1 Running 0 81s
```

part-d5-scaling-3-pods.png — kubectl get pods showing 3 flask-api pods

Figure 23: Flask API scaled to 3 replicas — all 3 pods in Running state

Rolling Update

A v2 image (*saadmughal009/flask-api:v2*) was built, pushed to DockerHub, and deployed using `kubectl set image`. Kubernetes performed a rolling update with zero downtime.


```
saad@saad-VirtualBox: ~/assi x + v
https://docs.docker.com/go/credential-store/
Login Succeeded
saad@saad-VirtualBox:~/assignment3/app/flask-api$ docker push saadmughal009/flask-api:v2
The push refers to repository [docker.io/saadmughal009/flask-api]
797d495f2c68: Layer already exists
45006ceeeea9: Layer already exists
1f5c2ee20069: Pushed
a84be4ab7c84: Pushed
2852dd905482: Pushed
4423fdd891c0: Pushed
8649771fee17: Layer already exists
e1b7436452c8: Pushed
5b4d6ff92fc4: Layer already exists
v2: digest: sha256:0ecb8578016504e78c7495774bb95cecff65c823248f881bea3f0fd5961ca7e0 size: 856
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl set image deployment/flask-api flask-api=saadmughal009/flask-a
pi:v2 -n assignment3
deployment.apps/flask-api image updated
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl rollout status deployment/flask-api -n assignment3
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
deployment "flask-api" successfully rolled out
saad@saad-VirtualBox:~/assignment3/app/flask-api$
```

part-d5-rolling-update.png — kubectl rollout status / pod update output

Figure 24: Rolling update in progress — pods updated to flask-api:v2

Rollback

A rollback was performed using `kubectl rollout undo deployment/flask-api -n assignment3`. Kubernetes reverted the deployment to the previous image (:latest) successfully.

```
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl rollout history deployment/flask-api -n assignment3
deployment.apps/flask-api
REVISION  CHANGE-CAUSE
1          <none>
2          <none>

saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl rollout undo deployment/flask-api -n assignment3
Warning: resource deployments/flask-api was previously managed with 'kubectl apply'. Rolling back will not update the kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/flask-api rolled back

saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl rollout status deployment/flask-api -n assignment3
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
deployment "flask-api" successfully rolled out

saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl describe deployment flask-api -n assignment3 | grep Image
Image:          saadmughal009/flask-api:latest
```

part-d5-rollback.png — kubectl rollout undo and get pods after rollback

Figure 25: Rollback completed — flask-api pods returned to :latest image

D6. End-to-End Testing & Verification

Curl Test via NodePort

The application was accessed externally at `http://$(minikube ip):30080`. Health endpoint and CRUD API (POST and GET /api/items) were tested through the full Nginx → Flask → MySQL path.

```

saad@saad-VirtualBox: ~/assi x + v
replicaset.apps/mysql-64d85fb859      1      1      1      3m5s
replicaset.apps/nginx-6bc4679456     1      1      1      59s
saad@saad-VirtualBox:~/assignment3$ kubectl get pv
NAME      CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCL
mysql-pv  1Gi       RWO           Retain          Bound   assignment3/mysql-pvc manual         <unset>
4m17s
saad@saad-VirtualBox:~/assignment3$ kubectl get pvc -n assignment3
NAME      STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
mysql-pvc Bound   mysql-pv  1Gi       RWO           manual         <unset>                4m23s
saad@saad-VirtualBox:~/assignment3$ kubectl get secrets -n assignment3
NAME      TYPE      DATA  AGE
mysql-secret  Opaque    4      5m49s
saad@saad-VirtualBox:~/assignment3$ kubectl get configmaps -n assignment3
NAME      DATA  AGE
flask-config  3      5m49s
kube-root-ca.crt  1      14m
nginx-conf  1      5m45s
saad@saad-VirtualBox:~/assignment3$ MINK_IP=$(minikube ip)
echo "App URL: http://${MINK_IP}:30080"
curl http://${MINK_IP}:30080/health
App URL: http://192.168.49.2:30080
{"service":"flask-api","status":"healthy"}
saad@saad-VirtualBox:~/assignment3$ curl -X POST http://${MINK_IP}:30080/api/items \
-H "Content-Type: application/json" \
-d '{"name": "Kubernetes Item", "description": "Created on K8s"}'
{"description":"Created on K8s","id":1,"name":"Kubernetes Item"}
saad@saad-VirtualBox:~/assignment3$ curl http://${MINK_IP}:30080/api/items
[{"created_at":"2026-06-01 08:16:22","description":"Created on K8s","id":1,"name":"Kubernetes Item"}]

```

part-d6-e2e-curl.png — curl health + POST + GET through NodePort

Figure 26: End-to-end curl test — health, POST item, GET items — all successful via :30080

Self-Healing

A Flask API pod was deliberately deleted using *kubectl delete pod*. Kubernetes automatically recreated it within seconds, demonstrating the ReplicaSet's self-healing capability.

```
operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/flask-api rolled back
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl rollout status deployment/flask-api -n assignment3
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "flask-api" rollout to finish: 1 old replicas are pending termination...
deployment "flask-api" successfully rolled out
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl describe deployment flask-api -n assignment3 | grep Image
Image:          saadmucha1000/flask-api:latest
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl get pods -n assignment3 -l app=flask-api
NAME                                READY   STATUS    RESTARTS   AGE
flask-api-7f56c46ddd-c86tm          1/1     Running   0           78m
flask-api-7f56c46ddd-ctkxb          1/1     Running   0           78m
flask-api-7f56c46ddd-m8m7q          1/1     Running   0           78m
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ FLASK_POD=$(kubectl get pod -l app=flask-api -n assignment3 -o jsonpat
h='{.items[0].metadata.name}')
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl delete pod ${FLASK_POD} -n assignment3
pod "flask-api-7f56c46ddd-c86tm" deleted from assignment3 namespace
saad@saad-VirtualBox: ~/assignment3/app/flask-api$ kubectl get pods -n assignment3 -l app=flask-api -w
NAME                                READY   STATUS    RESTARTS   AGE
flask-api-7f56c46ddd-ctkxb          1/1     Running   0           79m
flask-api-7f56c46ddd-m8m7q          1/1     Running   0           79m
flask-api-7f56c46ddd-nq9wp          1/1     Running   0           43s
```

part-d6-self-healing.png — pod terminating then new pod ContainerCreating → Running

Figure 27: Self-healing — deleted Flask pod immediately recreated by Kubernetes ReplicaSet

Database Persistence After MySQL Pod Deletion

The MySQL pod was deleted to simulate a failure. After Kubernetes recreated the pod, a GET request confirmed all previously inserted data was still present — proving the PersistentVolume correctly retains data independent of the pod lifecycle.

```

saad@saad-VirtualBox: ~/assi x + v
deployment "flask-api" successfully rolled out
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl describe deployment flask-api -n assignment3 | grep Image
Image:          saadmughal009/flask-api:latest
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl get pods -n assignment3 -l app=flask-api
NAME                                READY   STATUS    RESTARTS   AGE
flask-api-7f56c46ddd-c86tm          1/1     Running   0           78m
flask-api-7f56c46ddd-ctkxb          1/1     Running   0           78m
flask-api-7f56c46ddd-m8m7q          1/1     Running   0           78m
saad@saad-VirtualBox:~/assignment3/app/flask-api$ FLASK_POD=$(kubectl get pod -l app=flask-api -n assignment3 -o jsonpat
h='{.items[0].metadata.name}')
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl delete pod ${FLASK_POD} -n assignment3
pod "flask-api-7f56c46ddd-c86tm" deleted from assignment3 namespace
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl get pods -n assignment3 -l app=flask-api -w
NAME                                READY   STATUS    RESTARTS   AGE
flask-api-7f56c46ddd-ctkxb          1/1     Running   0           79m
flask-api-7f56c46ddd-m8m7q          1/1     Running   0           79m
^Csaad@saad-VirtualBox:~/assignment3/app/flask-api$ MINK_IP=$(minikube ip)
saad@saad-VirtualBox:~/assignment3/app/flask-api$ curl http://${MINK_IP}:30080/api/items
[{"created_at":"2026-06-01 08:16:22","description":"Created on K8s","id":1,"name":"Kubernetes Item"}]
saad@saad-VirtualBox:~/assignment3/app/flask-api$ MYSQL_POD=$(kubectl get pod -l app=mysql -n assignment3 -o jsonpath='{
.items[0].metadata.name}')
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl delete pod ${MYSQL_POD} -n assignment3
pod "mysql-64d85fb859-442l9" deleted from assignment3 namespace
saad@saad-VirtualBox:~/assignment3/app/flask-api$ kubectl get pods -n assignment3 -l app=mysql -w
NAME                                READY   STATUS    RESTARTS   AGE
mysql-64d85fb859-fsgh4              1/1     Running   0           55s
^Csaad@saad-VirtualBox:~/assignment3/app/flask-api$ curl http://${MINK_IP}:30080/api/items
[{"created_at":"2026-06-01 08:16:22","description":"Created on K8s","id":1,"name":"Kubernetes Item"}]

```

part-d6-persistence-proof.png — MySQL pod deleted + recreated + data still present

Figure 28: Data persistence — MySQL pod deleted and recreated, all items still returned by API

YAML Manifest Explanations

This section provides a brief explanation of each Kubernetes manifest file used in the project.

1. namespace.yml

Defines the Kubernetes namespace *assignment3* to logically isolate all project resources from other workloads on the cluster. All subsequent resources are deployed into this namespace using the *-n assignment3* flag.

2. mysql-secret.yml

Creates a Kubernetes Secret of type *Opaque* named *mysql-secret*. Stores Base64-encoded MySQL credentials: root password, application username, and application password. Both the MySQL and Flask deployments reference this secret via environment variable injection — no credentials are hardcoded in the image or deployment YAML.

3. flask-configmap.yml

Creates a ConfigMap named *flask-config* holding non-sensitive Flask configuration: the MySQL service hostname (*mysql*), database name, and port (3306). The Flask deployment reads these values as environment variables, allowing configuration changes without rebuilding the image.

4. nginx-configmap.yml

Creates a ConfigMap named *nginx-config* containing the Nginx reverse proxy configuration. It defines upstream routing rules that forward all HTTP requests at port 80 to the Flask API service at port 5000. The Nginx deployment mounts this ConfigMap as a volume at */etc/nginx/conf.d/*.

5. mysql-pv.yml

Defines a PersistentVolume (PV) named *mysql-pv* with 1Gi capacity, ReadWriteOnce access mode, and hostPath storage at */data/mysql* on the Minikube node. The Retain reclaim policy ensures data is not deleted when the PVC is released.

6. mysql-pvc.yml

Creates a PersistentVolumeClaim (PVC) named *mysql-pvc* in the *assignment3* namespace. It requests 1Gi of storage with ReadWriteOnce access, which binds to the *mysql-pv* PV. The MySQL deployment references this PVC to mount persistent storage at */var/lib/mysql*.

7. mysql-deployment.yml

Defines the MySQL Deployment with 1 replica using the *mysql:8.0* image. Credentials are injected from *mysql-secret*. The */var/lib/mysql* directory is mounted from *mysql-pvc* for data persistence. A liveness probe checks the MySQL process health.

8. mysql-service.yml

Exposes MySQL internally using a ClusterIP service on port 3306. The selector targets pods with label *app: mysql*. ClusterIP ensures MySQL is only reachable within the cluster by the Flask API — it is never exposed to external traffic.

9. flask-deployment.yml

Defines the Flask API Deployment using *saadmughal009/flask-api:latest*. Database settings are injected from *flask-config* (ConfigMap) and *mysql-secret* (Secret). A readiness probe on */health* ensures traffic is only routed to ready pods. Scaled to 3 replicas in D5.

10. flask-service.yml

Exposes Flask API using a ClusterIP service on port 5000. Nginx routes HTTP requests to this service by name (*flask-api:5000*). ClusterIP keeps Flask internal — all external clients must go through Nginx, maintaining 3-tier separation.

11. nginx-deployment.yml

Defines the Nginx Deployment with 1 replica using *saadmughal009/nginx-proxy:latest*. The *nginx-config* ConfigMap is mounted as a volume at */etc/nginx/conf.d/default.conf*. Nginx acts as the single entry point for all external traffic.

12. nginx-service.yml

Exposes Nginx externally using a NodePort service. Port 80 inside the container is mapped to NodePort *30080* on the Minikube node's IP. This allows external access at *http://\$(minikube ip):30080* — the only externally accessible entry point to the application.

Conclusion

This assignment successfully implemented a complete, production-grade DevOps pipeline from source code management to containerized deployment on a Kubernetes cluster. Each stage was completed and verified:

- **Part A - Git:** Repository initialized with a proper branching strategy (main, develop, feature/*), 10+ descriptive commits, .gitignore, and GitHub remote hosting.
- **Part B - Docker:** Flask API and Nginx containerized using custom Dockerfiles. Docker Compose orchestrated the 3-tier stack with a custom bridge network and named volume for MySQL data persistence.
- **Part C - CI/CD:** GitHub Actions automatically builds and pushes Docker images to DockerHub on every push to main, using encrypted secrets for credential management with no hardcoded values.
- **Part D - Kubernetes:** Full deployment on Minikube with namespaces, PV/PVC persistent storage, Secrets, ConfigMaps, scaling to 3 replicas, rolling update, rollback, self-healing, and end-to-end API verification through NodePort 30080.

The project demonstrates a real-world DevOps workflow where infrastructure is code, deployments are automated, and applications are resilient to failure. Kubernetes-native features — ReplicaSets for self-healing, PersistentVolumes for data durability, Secrets for credential management, and ConfigMaps for environment-specific configuration — reflect modern cloud-native best practices.

GitHub Repository: <https://github.com/TheIllumi/assignment3-devops>

DockerHub: <https://hub.docker.com/u/saadmughal009>